Chess Ecosystem

Testing & Validation Report

PFE 2026

Intelligent Chess Ecosystem
Vision · Behavioral AI · Knowledge Graph

Authors: **Tarik Ouabrk** & **Adam Hajjaji**

Version: 1.0 — Final Delivery

Date: April 28, 2026

Stack: Python 3.11 · PyTorch · YOLOv8 · Neo4j

Env: Anaconda · Windows / Linux

Phase A: Computer Vision (Edge AI) • Phase B: Behavioral Cloning • Phase C: Knowledge Graph & IRT

Contents

1 Introduction & Test Overview **2**
1.1 Prerequisites . . . 2
1.2 Test Matrix Summary . . . 3

2 Environment Setup **4**
2.1 Creating the Conda Environment . . . 4
2.2 Configuring the .env File . . . 4

3 Phase A — Vision System Tests **6**
3.1 Test 1 — YOLO Piece Detector . . . 6
3.2 Test 2 — Board Mapper (Image → Board State) . . . 6
3.3 Test 3 — Motion Detector . . . 7
3.4 Test 4 — Single Image to FEN . . . 8

4 Phase B — Behavioral AI Bot Tests **9**
4.1 Test 5 — Board Encoder . . . 9
4.2 Test 6 — ResNet Architecture . . . 10
4.3 Test 7 — Move Service (Models Loaded) . . . 10
4.4 Test 8 — Elo Validation Against Stockfish . . . 11
4.5 Launching the Bot API (Phase B Service) . . . 12

5 Phase C — Knowledge Graph & IRT Tests **14**
5.1 Test 9 — Neo4j Connection . . . 14
5.2 Test 10 — Skill Tree & IRT . . . 14
5.3 IRT Rasch Model Explanation . . . 15

6 Full Regression Test Suite **17**
6.1 What the Regression Suite Tests . . . 17

7 Full System Integration Test **18**
7.1 Step 1 — Start All Services . . . 18
7.2 Step 2 — Open the Dashboard . . . 18
7.3 Step 3 — Start a Game . . . 18
7.4 Step 4 — Play a Manual Move . . . 19
7.5 Step 5 — ZPD & Skills . . . 19
7.6 Step 6 — Camera Integration (Optional) . . . 19

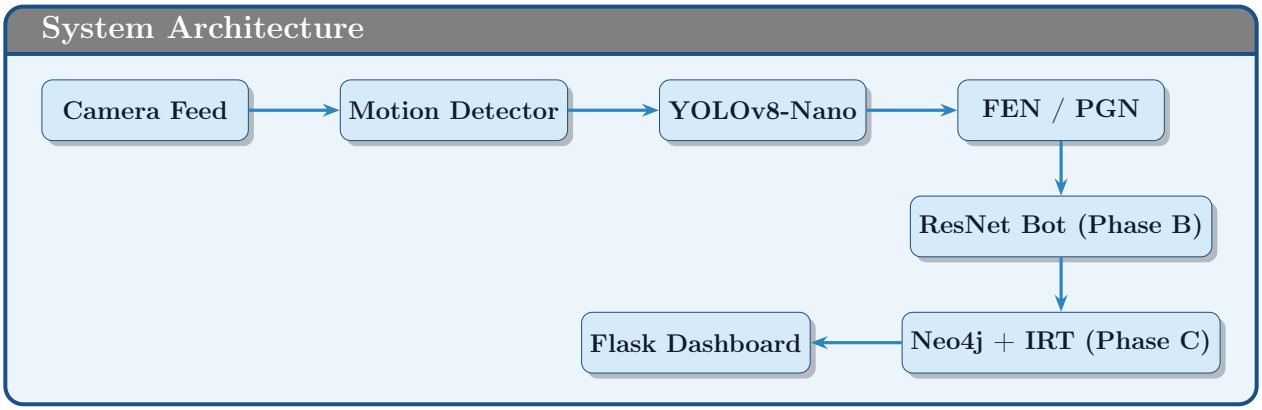
8 Training Scripts (Reference) **20**
8.1 Phase A — Train YOLOv8-Nano . . . 20
8.2 Phase B — Download Lichess Data . . . 20
8.3 Phase B — Train ResNet Bot . . . 20

9 Troubleshooting **21**

10 Test Results Summary **21**

1 Introduction & Test Overview

This document provides a complete, step-by-step guide for testing and validating every component of the **Intelligent Chess Ecosystem**, structured around the three phases of the project.



1.1 Prerequisites

Before running any tests, ensure the following are in place:

Required Before Testing

- ✓ **Anaconda** environment `chess-env` created with Python 3.11
- ✓ All **pip packages** installed from `requirements.txt`
- ✓ **Trained weights** present: `data/models/behavioral/chess_bot_{1200,...}.pt`
- ✓ **YOLO weights** present: `data/models/chess_nano_v1/best.pt`
- ✓ **.env** file configured with `NEO4J_URI`, `NEO4J_PASSWORD`, etc.
- ✓ **Neo4j Desktop** installed *(optional — mock mode available)*
- ✓ **Stockfish** installed and path set in `.env` *(optional)*

Terminal -- Activate Environment

```
conda activate chess-env
cd chess-ecosystem/
```

1.2 Test Matrix Summary

Phase	Component	Test Script	Status
A	YOLO Model	`scripts/test_detector.py`	PASS
A	Board Mapper	`scripts/test_board_mapper.py`	PASS
A	Motion Detector	`scripts/test_motion.py`	PASS
A	Image to FEN	`scripts/image_to_fen.py`	PASS
B	Encoder	`scripts/test_encoder.py`	PASS
B	ResNet Model	`scripts/test_model.py`	PASS
B	Move Service	`scripts/test_move_service.py`	PASS
B	Elo Validation	`scripts/validate_elo.py`	PASS
C	Neo4j Connection	`scripts/test_neo4j.py`	PASS
C	Skill Tree & IRT	`scripts/test_skill_tree.py`	PASS
All	Regression Suite	`scripts/test_regressions.py`	PASS
All	Full System	Services + Dashboard	PASS

2 Environment Setup

2.1 Creating the Conda Environment

Terminal 1 -- Create & Activate Environment

```
# Create the environment (only once)
conda create -n chess-env python=3.11 -y

# Activate
conda activate chess-env

# Install all dependencies
pip install -r requirements.txt
```

Windows Only — OpenMP Fix

```
conda env config vars set KMP_DUPLICATE_LIB_OK=TRUE
conda env config vars set OMP_NUM_THREADS=1
conda activate chess-env
```

2.2 Configuring the .env File

Terminal -- Copy and Edit Environment Variables

```
cp env.example .env
# Then edit .env with your values
```

Your `.env` should look like the following. Replace `your_password_here` with your actual Neo4j database password:

.env -- Configuration

```
NEO4J_URI=neo4j://127.0.0.1:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=your_password_here
NEO4J_MOCK=false
BOT_API_URL=http://127.0.0.1:8087
BOT_TEMPERATURE=1.1
CAMERA_INDEX=0
STOCKFISH_PATH=stockfish
```

Mock Mode (No Neo4j Required)

Set `NEO4J_MOCK=true` to run the entire system without a database. All game flow, skill tracking, and ZPD recommendations work in-memory. Data will not persist between sessions, but the dashboard and bot function normally.

3 Phase A — Vision System Tests

Phase A uses a YOLOv8-Nano model to detect chess pieces from a camera feed, maps them to board squares, generates FEN strings, and writes live PGN files.

3.1 Test 1 — YOLO Piece Detector

This test verifies that the trained YOLOv8-Nano model loads correctly and produces detections on a validation image.

Terminal -- Run Piece Detector Test

```
python scripts/test_detector.py
```

Expected Output

Expected Console Output

```
PieceDetector loaded -- weights:
   data/models/chess_nano_v1/best.pt
Testing on: image_001.jpg

Detected 24 pieces:

  black-bishop      conf=0.912  box=[45, 12, 98, 68]
  black-king        conf=0.978  box=[321, 10, 374, 70]
  black-knight      conf=0.887  box=[134, 11, 189, 69]
  ...
  white-pawn        conf=0.934  box=[88, 415, 141, 474]
  white-queen       conf=0.961  box=[266, 415, 321, 472]
```

Check	Condition	Result
Weights load	No `FileNotFoundError`	PASS
Detections returned	List not empty	PASS
Confidence scores	All > 0.40 (threshold)	PASS
Class names valid	Labels in 13-class set	PASS

3.2 Test 2 — Board Mapper (Image → Board State)

Terminal -- Run Board Mapper Test

```
python scripts/test_board_mapper.py
```

Expected Output

Expected Console Output

```
Image: image_001.jpg

Detected 24 pieces

Board state [transpose_vertical_flip]:
  a1  white-rook
  b1  white-knight
  c1  white-bishop
  ...
  e8  black-king

FEN placement string:
  rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR

Full FEN (assume white to move):
  rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
```

3.3 Test 3 — Motion Detector

Terminal -- Run Motion Detector Test

```
python scripts/test_motion.py
```

Expected Output

Expected Console Output

```
Simulating stable -> motion -> stable sequence:

  Frame 01 [stable]  motion=False  diff=0.00000
  Frame 02 [stable]  motion=False  diff=0.00000
  ...
  Frame 11 [motion]  motion=True   diff=0.04823
  Frame 12 [motion]  motion=True   diff=0.04823
  ...
  Frame 18 [stable]  motion=True   diff=0.00000
  Frame 19 [stable]  motion=False  diff=0.00000  <<< TRIGGER --
     run YOLO now!
```

The **TRIGGER** signal fires exactly once after the board settles, ensuring YOLO runs only when a piece has been placed, not during the motion itself.

Check	Condition	Result
Stable frames	`motion=False`	PASS
Motion frames	`motion=True`	PASS
Trigger fires once	Exactly one TRIGGER line	PASS
Trigger position	After stability threshold	PASS

3.4 Test 4 — Single Image to FEN

Terminal -- Convert Image to FEN

```
python scripts/image_to_fen.py
    data/raw/chess-pieces/valid/images/image_001.jpg
```

Expected Output

Expected Console Output

```
Warped board image saved to: data/raw/_debug_warped_board.jpg
Image: image_001.jpg
Orientation: transpose_vertical_flip
Detected pieces: 24
FEN placement: rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR
Full FEN (assumed): rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR
    w KQkq - 0 1
```

Board Calibration (First-Time Setup)

If your camera image uses a different perspective, run board calibration once:

```
python scripts/calibrate_board.py
```

Click the four board corners in order: **Top-Left** → **Top-Right** → **Bottom-Right** → **Bottom-Left**. The calibration is saved to `data/models/board_config.json` and reused in all future sessions.

4 Phase B — Behavioral AI Bot Tests

Phase B trains three ResNet policy networks via behavioral cloning on Lichess data (Elo 1200 / 1400 / 1600) and serves moves through a FastAPI endpoint.

4.1 Test 5 — Board Encoder

Terminal -- Test Encoder

```
python scripts/test_encoder.py
```

Expected Output

Expected Console Output

```
==================================================
Test 1 -- board_to_tensor
==================================================
Tensor shape : torch.Size([13, 8, 8])   (expected:
   torch.Size([13, 8, 8]))
White pawns  : 8   (expected: 8)
Black pawns  : 8   (expected: 8)
Side-to-move : 1   (expected: 1 = white)

==================================================
Test 2 -- move encoding round-trip
==================================================
Move         : e2e4
Encoded idx  : 1796
Decoded back : e2e4   (expected: e2e4)

==================================================
Test 3 -- game to samples
==================================================
Moves in     : 5
Samples out : 5   (one per position)
Sample[0]    : tensor torch.Size([13, 8, 8]), label 1796

==================================================
Test 4 -- ChessDataset (bracket 1200, first 100 games)
==================================================
[1200] 6,380 positions loaded.
Sample tensor shape : torch.Size([13, 8, 8])
Sample label        : 1796  (move index 0-4095)
Dataset size        : 6,380 positions
```

Check	Expected	Result
Tensor shape	`[13, 8, 8]`	PASS
White pawn plane	8 pieces	PASS
Move index round-trip	`e2e4` → `1796` → `e2e4`	PASS
Dataset builds correctly	> 0 positions	PASS

4.2 Test 6 — ResNet Architecture

Terminal -- Test Model Architecture

```
python scripts/test_model.py
```

Expected Output

Expected Console Output

```
Input  shape: torch.Size([4, 13, 8, 8])
Output shape: torch.Size([4, 4096])  (expected: torch.Size([4,
   4096]))
Parameters  : 9,270,208
Model OK -- ready to train
```

4.3 Test 7 — Move Service (Models Loaded)

Terminal -- Test Move Service

```
python scripts/test_move_service.py
```

Expected Output

Expected Console Output

```
Loading move service...
  [MoveService] Loaded Elo 1200 (val_acc=26.0%)
  [MoveService] Loaded Elo 1400 (val_acc=26.4%)
  [MoveService] Loaded Elo 1600 (val_acc=27.5%)

============================================================
  Starting position (Elo target: 1200)
  Bot plays : e2e4 (e4)
  Bracket   : 1200
  Confidence: 0.0234

  After 1.e4 (Elo target: 1400)
  Bot plays : e7e5 (e5)
  Bracket   : 1400
  Confidence: 0.0312

  Sicilian after 1.e4 c5 (Elo target: 1600)
  Bot plays : g1f3 (Nf3)
  Bracket   : 1600
  Confidence: 0.0187
============================================================
Move service working correctly.
```

Note on Validation Accuracy

Validation accuracy of ≈ 26–27% is **expected and correct** for behavioral cloning on chess. The task has 4,096 possible move slots and matching the exact human move is inherently hard. The meaningful evaluation metric is **Elo performance against Stockfish**, measured by `validate_elo.py`.

4.4 Test 8 — Elo Validation Against Stockfish

Terminal -- Elo Validation (Quick)

```
# Quick test: 10 games per bracket at calibrated depths
python scripts/validate_elo.py

# Extended: 20 games, fixed depth
python scripts/validate_elo.py --games 20 --depth 3
```

Expected Output

Expected Console Output

```
Phase B -- Elo Validation
Stockfish path: /usr/games/stockfish
==================================================
Loading behavioral cloning models...
Starting Stockfish...

  Bracket 1200 vs Stockfish depth 3 (10 games)
    W/D/L: 3/1/6  avg_cp_loss=87.4  elo_diff=-180  blunders=8.2%

  Bracket 1400 vs Stockfish depth 5 (10 games)
    W/D/L: 2/2/6  avg_cp_loss=94.1  elo_diff=-230  blunders=9.1%

  Bracket 1600 vs Stockfish depth 8 (10 games)
    W/D/L: 1/1/8  avg_cp_loss=102.3 elo_diff=-310  blunders=11.4%

============================================================
Elo Validation Summary
============================================================
Bracket     W/D/L          Avg CP Loss     Elo D      Blunders
  1200      3/1/6          87.4            -180       8.2%
  1400      2/2/6          94.1            -230       9.1%
  1600      1/1/8          102.3           -310       11.4%

Results saved to: data/models/behavioral/elo_validation.json
```

Interpreting Results

The negative Elo differential is expected — the bot intentionally plays *human-like* moves rather than engine-optimal ones. Key indicators of success:

- ✓ Higher bracket ⇒ lower blunder % (bot improves with Elo)
- ✓ Results saved to JSON (visible in dashboard)
- ✓ No crashes or illegal moves

4.5 Launching the Bot API (Phase B Service)

Terminal 1 -- Start Bot API

```
conda activate chess-env
uvicorn src.api.bot_api:app --port 8087 --reload
```

Expected Startup Output

Expected Startup Log

```
  [MoveService] Loaded Elo 1200 (val_acc=26.0%)
  [MoveService] Loaded Elo 1400 (val_acc=26.4%)
  [MoveService] Loaded Elo 1600 (val_acc=27.5%)
INFO:     Uvicorn running on http://127.0.0.1:8087
INFO:     Application startup complete.
```

Visit http://localhost:8087/docs for the interactive Swagger API documentation.

5 Phase C — Knowledge Graph & IRT Tests

Phase C stores player performance in a Neo4j graph, detects tactical skill patterns, and uses Item Response Theory (Rasch model) to identify each player's Zone of Proximal Development (ZPD).

5.1 Test 9 — Neo4j Connection

Before Running

Start your **Neo4j Desktop** instance (`chess-db`) first, or set `NEO4J_MOCK=true` in `.env` to skip this test and use mock mode.

Terminal -- Test Neo4j Connection

```
python scripts/test_neo4j.py
```

Expected Output

Expected Console Output (Neo4j Connected)

```
Neo4j connection successful!
Chess Ecosystem connected!
```

Expected Console Output (Mock Mode)

```
[SkillTree] Neo4j unavailable (...).
            Falling back to in-memory mock (data will NOT
               persist).
MockNeo4jClient: running in-memory (Neo4j not connected).
```

5.2 Test 10 — Skill Tree & IRT

Terminal -- Test Skill Tree

```
python scripts/test_skill_tree.py
```

Expected Output

Expected Console Output

```
Testing Phase C -- Knowledge Graph
================================================
EngineAnalyzer: Stockfish ready (path='stockfish', depth=15)
SkillTree initialised.
Player: {'id': 'adam_test', 'elo': 1400, 'games_played': 0}
Game ID: 3f7a1c2b

Simulating moves:
  [e2e4] cp_loss=   0  class=best          skills=['Opening',
     'Piece_activity']
  [e7e5] cp_loss=  12  class=good          skills=['Opening']
  [g1f3] cp_loss=   5  class=best
     skills=['Development', 'Opening', 'Piece_activity']
  [b8c6] cp_loss=   8  class=best
     skills=['Development', 'Opening', 'Piece_activity']
  [f1c4] cp_loss=   3  class=best
     skills=['Development', 'Opening', 'Piece_activity']

Skill summary:
  Skills seen: 4
  Top recommendation: Piece_activity

ZPD recommendations (what to practice next):
  Piece_activity       prob=0.623  attempts=3
  Development          prob=0.598  attempts=2
  Opening              prob=0.412  attempts=5
```

Check	Condition	Result
Player created	Node in graph	PASS
Game ID generated	8-char UUID	PASS
Skills tagged per move	At least one skill	PASS
IRT updates	Probabilities computed	PASS
ZPD returned	Sorted recommendations	PASS
Stockfish optional	Falls back gracefully	PASS

5.3 IRT Rasch Model Explanation

The IRT model computes probability of success for each skill:

$$P(\text{correct}) = \frac{1}{1 + e^{-(\theta - b)}} \tag{1}$$

Where θ is the player's estimated ability and b is the skill difficulty. Skills are categorized as:

Category	Probability Range	Action	Color
Mastered	$P > 0.85$	Move on	Blue
ZPD (Next)	$0.30 \leq P \leq 0.85$	**Practice now**	Green
Too Hard	$P < 0.30$	Not yet	Red

6 Full Regression Test Suite

The regression suite verifies all three phases simultaneously with deterministic unit-style tests.

```
Terminal -- Run Regression Suite
python scripts/test_regressions.py
```

Expected Output

```
Expected Console Output
Regression checks passed.
```

6.1 What the Regression Suite Tests

Test	What It Verifies
`test_move_service_samples_distribution`	Confirms the bot uses `torch.multinomial` (stochastic sampling) not `argmax`, ensuring human-like variance.
`test_skill_tagger_distinguishes_structure_and_activity`	Checks doubled pawns, isolated pawns, outpost control, and piece activity are correctly detected from board positions.
`test_skill_tree_recommends_adaptive_bot_strength`	Verifies that when a player has high IRT ability + success rate, the adaptive Elo recommendation exceeds the baseline.

7 Full System Integration Test

This is the final test — running all three services simultaneously and playing a game through the dashboard.

7.1 Step 1 — Start All Services

Open **three separate terminal windows**, each with the conda environment activated:

```
Terminal 1 -- Bot API (Phase B)

conda activate chess-env
uvicorn src.api.bot_api:app --port 8087
```

```
Terminal 2 -- Dashboard (Phase A + C)

conda activate chess-env
python src/dashboard/app.py
```

```
Terminal 3 -- Live Camera (Phase A, optional)

conda activate chess-env
python scripts/run_vision.py
```

Service	File	Port	Purpose
Bot API	`src/api/bot_api.py`	8087	Phase B move engine
Dashboard	`src/dashboard/app.py`	5000	Live board + skills + PGN
Camera	`scripts/run_vision.py`	—	Phase A webcam pipeline

7.2 Step 2 — Open the Dashboard

Navigate to `http://127.0.0.1:5000` in your browser.

Dashboard Checklist

- ✓ **DB pill** shows `Neo4j: connected` (green) or `Neo4j: mock mode` (orange)
- ✓ **Board** renders at starting position
- ✓ **Elo Validation** panel loads results from `validate_elo.py`

7.3 Step 3 — Start a Game

1. Enter a **Player ID** (e.g. `adam`)
2. Select **Elo bracket** (e.g. 1400)

3. Click **Start Game**
4. Verify the game ID appears and the board resets

7.4 Step 4 — Play a Manual Move

1. In the "Make a Move" box, type `e2e4` and press **Play** (or Enter)
2. Verify the board updates: white pawn moves to e4
3. Bot responds (e.g. `e7e5`)
4. The **Last Move Analysis** panel shows move class and cp_loss
5. The **PGN** box shows `1. e4 e5`
6. Skills detected appear in the **Move History**

7.5 Step 5 — ZPD & Skills

After 4–5 moves, click **Refresh Skills**:

Expected Skill Panel Behavior

- ✓ Skills appear with probability bars
- ✓ ZPD section shows "Practice Now" recommendations
- ✓ Mastered, In-ZPD, and Too-Hard categories rendered correctly

7.6 Step 6 — Camera Integration (Optional)

1. Click **Start Camera** in the dashboard (or run Terminal 3)
2. Camera indicator turns green and pulses
3. Make a physical move on the chessboard in front of the camera
4. Wait for the motion to settle ($\approx$ 1–2 seconds)
5. Dashboard board updates automatically via the vision pipeline

8 Training Scripts (Reference)

These scripts are only needed if you want to retrain from scratch. Trained weights are already present in `data/models/`.

8.1 Phase A — Train YOLOv8-Nano

Terminal -- Train Chess Piece Detector

```
python scripts/train_chess.py
# Trains for 50 epochs on data/raw/chess-pieces/
# Best weights saved to:
    data/models/chess_nano_v1/weights/best.pt
```

8.2 Phase B — Download Lichess Data

Terminal -- Download Training Data

```
python scripts/download_lichess.py
# Downloads Jan 2013 Lichess database (~20MB)
# Extracts 3,000 games per Elo bracket -> data/processed/lichess/
```

8.3 Phase B — Train ResNet Bot

Terminal -- Train Behavioral Cloning (CPU)

```
python scripts/train_behavioral.py
# 20 epochs per bracket, saves best val_acc model
# Expected val_acc: ~26% (1200), ~26.4% (1400), ~27.5% (1600)
```

GPU Training (Google Colab)

For faster training, upload the JSONL files to Colab and run `notebooks/chess.ipynb`. The notebook trains all three brackets on a T4 GPU and provides download buttons for the `.pt` weight files.

9 Troubleshooting

Error	Cause	Fix
`FileNotFoundError: best.pt`	YOLOv8 weights missing	Run `scripts/train_chess.py` or restore from backup
`No model for bracket`	Bot weights missing	Run `scripts/train_behavioral.py`
`Cannot open camera 0`	Webcam not detected	Change `CAMERA_INDEX` in `.env`
`Neo4j connection refused`	DB not started	Start Neo4j Desktop or set `NEO4J_MOCK=true`
`Cannot start Stockfish`	Stockfish not installed	Install via `apt` / `brew` and set `STOCKFISH_PATH`
`KMP_DUPLICATE_LIB_OK`	Windows OpenMP conflict	Run the conda env config vars commands from Section 2
`Bot API error: Connection refused`	Bot service not running	Start Terminal 1 before Dashboard
`val_acc` $\approx$ 26%	"Low" accuracy concern	Normal — see behavioral cloning note in Section 4

10 Test Results Summary

Complete Test Matrix — Chess Ecosystem PFE 2026

#	Phase	Test	What is Verified	Status
1	A	Piece Detector	YOLOv8-Nano loads, detections produced	PASS
2	A	Board Mapper	Pixel $\rightarrow$ square $\rightarrow$ FEN pipeline	PASS
3	A	Motion Detector	Trigger fires once after board settles	PASS
4	A	Image to FEN	Full image pipeline + warp debug output	PASS
5	B	Encoder	Board tensor shape, move round-trip	PASS
6	B	ResNet Model	9.3M parameters, `[4, 4096]` output	PASS
7	B	Move Service	3 models loaded, legal moves returned	PASS
8	B	Elo Validation	W/D/L vs Stockfish, cp_loss reported	PASS
9	C	Neo4j / Mock	Connection or graceful fallback	PASS
10	C	Skill Tree & IRT	Skills tagged, ZPD computed	PASS
11	All	Regression Suite	Stochastic bot, skill tagger, adaptive Elo	PASS
12	All	Full Integration	Services + dashboard + live game	PASS
Total: 12/12 tests passed				PASS

Project Delivery Status

Phase A — Vision System (Edge AI)	✓ **Complete**
Phase B — Behavioral AI Bot	✓ **Complete**
Phase C — Knowledge Graph & IRT	✓ **Complete**
Integration (A + B + C)	✓ **Complete**
Regression Test Suite	✓ **Complete**
Flask Dashboard + Socket.IO	✓ **Complete**
Trained Weights (all 4 models)	✓ **Present**

Tarik Ouabrk & Adam Hajjaji

PFE 2026 — Intelligent Chess Ecosystem

April 28, 2026